System Verification and Validation Plan for TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

November 1, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
  - 2.1.1 User Interface . . . 1
  - 2.1.2 Audio Analysis Tool (SAFA) . . . 2
  - 2.1.3 Report Generation . . . 2
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 3

3 Plan **4**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 4
- 3.3 Design Verification . . . 4
- 3.4 Verification and Validation Plan Verification . . . 4
- 3.5 Implementation Verification . . . 5
- 3.6 Automated Testing and Verification Tools . . . 5
- 3.7 Software Validation . . . 6

4 System Tests **6**
- 4.1 Tests for Functional Requirements . . . 6
  - 4.1.1 Area of Testing1 . . . 6
  - 4.1.2 Data Collection and Storage . . . 7
  - 4.1.3 Video and Audio Analysis . . . 11
- 4.2 Tests for Nonfunctional Requirements . . . 15
  - 4.2.1 Area of Testing1 . . . 15
  - 4.2.2 Accuracy . . . 16
  - 4.2.3 Reliability . . . 17
  - 4.2.4 Recording Quality . . . 18
  - 4.2.5 Maintainability . . . 19
  - 4.2.6 Traceability Between Test Cases and Requirements . . . 21

5 Unit Test Description **21**
- 5.1 Unit Testing Scope . . . 21
- 5.2 Tests for Functional Requirements . . . 22

5.2.1 Module 1 . . . 22
5.2.2 Module 2 . . . 23
5.3 Tests for Nonfunctional Requirements . . . 23
5.3.1 Module ? . . . 23
5.3.2 Module ? . . . 24
5.4 Traceability Between Test Cases and Modules . . . 24

6 Appendix **25**
6.1 Symbolic Parameters . . . 25
6.2 Usability Survey Questions? . . . 25

7 Appendix — Generative AI Use Disclosure **26**
7.1 AI Tools Used . . . 26
7.2 How and Where Used . . . 26
7.3 Verification of AI-Generated Content . . . 27

8 Appendix — Reflection **28**

List of Tables

1 Variable Names and Values . . . 25
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
SAFA	Stands for Speech Annotation For Application, Access, Accuracy
SLP	Speech Language Pathologist
TeLaC	Telehealth Language Assessment for Children

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Mudassar et al., 2026) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

This section provides a brief overview of the software being produced and its objectives, how it will be tested and its relevant documentation.

2.1 Summary

The software being tested is TeLaC, a platform for remote speech and language assessment of Mandarin-English bilingual children. It addresses the usability and workflow challenges speech-language pathologists (SLPs) face when conducting remote assessments of bilingual pediatric patients. It does this by integrating the SAFA annotation tool into one platform, auto-generating transcripts, providing a hub for patient records and assessment results, and generating assessment reports. The goal is to help clinicians conduct assessments more efficiently and consistently while supporting ongoing monitoring of a child's speech development, all in one centralized system.

The objective of this VnV plan is to build confidence that TeLaC is usable, accurate, and trustworthy enough to support clinical decisions. TeLaC assists clinicians and does not replace their judgment (SR3). The requirements being verified are defined in the SRS, and the risks that determine test priority are identified in the Hazard Analysis. The system has three main components described below.

2.1.1 User Interface

The user interface must be intuitive and easy to navigate for clinicians with limited technical fluency (SR8, NFR1, NFR2). SLPs use it for the entire workflow, from selecting a patient to downloading a report (FR5, FR9). A confusing interface slows them down and risks mistakes, so the plan verifies the following:

- Clinicians can complete core tasks, such as assigning a speaker role, viewing results, and adding a patient, without training (NFR1)
- Clinicians can move between workflow stages without losing saved work (FR5, NFR3)

- Clinicians can always tell which content is automatically generated and which they have verified (FR14).

2.1.2 Audio Analysis Tool (SAFA)

SAFA auto-generates transcripts of the assessment recordings and identifies speakers (FR1, FR4). Clinicians then verify and correct these outputs (FR2, FR6). Transcript accuracy is critical considering errors in code-mixed Mandarin-English speech, can lead to wrong conclusions about a child's abilities (SRS Section 9.4). The system is therefore tested to ensure that:

- Transcripts are generated reliably for supported videos (FR1, NFR17)
- The transcript stays synchronized with the recording during playback (FR20, FR21, NFR13)
- Clinician edits and annotations are saved and persist across sessions (FR2, FR19, NFR18)

2.1.3 Report Generation

Report generation compiles a patient's results, the clinician's edits, and notes into a report that the SLP can review and edit before downloading (FR7, SR6). Because the report becomes part of the clinical record and may be relied on by families, schools, and other professionals, its correctness is crucial. The system is therefore tested to ensure that:

- Every change a clinician makes must appear in the report, otherwise the automatically generated content remains as is (FR7)
- Edits to results are reflected correctly and instantly (FR8, NFR4)
- Exports and reports are only accessible to the assigned clinician (NFR5, SRS Section 7.5)

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate

adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Mudassar et al. (2026)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]

[The review will include reviews by your classmates —SS]

[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walkthroughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walkthrough. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

The following lists the automated testing and verification tools used throughout the software's lifecycle:

- Unit Tests: pytest (Python), Jest (TypeScript/JavaScript)
- Linters: Ruff (Python), ESLint (TypeScript/JavaScript)
- Continuous Integration: GitHub Actions

The CI workflows and tooling are described further in the described further in the Development Plan.

For code coverage, Coverage.py is used for the Python code. For TypeScript/JavaScript, Istanbul is used. This is considering it is built into Jest, which reduces the number of additional frameworks needed. Both tools report which lines and functions are not covered by the tests, so the team can see where additional tests are needed. The team's target is approximately 80% for line coverage, considering that is the current standard. Furthermore, it keeps the testing effort balanced with the time available for developing the software.

Ruff is used to ensure the Python coding standard (PEP 8) is respected. It was chosen for its speed compared to alternative linters. Additionally, since TypeScript/JavaScript makes up a large part of the codebase, ESLint is used to find and report problematic patterns and bugs in that code. ESLint is the industry standard for static analysis of JavaScript, which is why it was chosen. Moreover, GitHub Actions runs the unit tests and the linting checks on every pull request that is opened or updated and on every merge to main. A pull request with failing checks must be fixed before it can be merged.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Data Collection and Storage

This section verifies that annotations are saved to the patient's assessment record, can be exported, and that recordings can be marked as reviewed.

1. FR-ST-DCS1: **Store Completed Annotations in the Assessment Record**

Purpose: Verifies that the system stores a clinician's saved transcript annotations in the patient's assessment record.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation interface with its auto-generated transcript.

Input: 3 assessment videos. For each video, 3 edits will be made to the transcript and documented with their corresponding chunks for later reference.

Output: After saving, every edit appears in that patient's assessment record when opened, and still appears after logging out and back in.

Test Case Derivation: FR19 requires that a saved annotation appear in the patient's assessment record from any clinician view without any file upload or download. This keeps patient information accurate and up to date and shortens the speech analysis workflow.

How the test will be performed:

1. In the annotation interface, make the listed edits to the transcript and save.
2. Open the assessment from the patient record and check that each edit is shown.
3. Log out, log back in, and check that the edits are still shown.

2. FR-ST-DCS2: **Export Annotation Data**

Purpose: Verifies that the system exports a video assessment's approved annotations in each supported file format, and that the exported file matches the stored annotations.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation

interface with its auto-generated transcript.

Input: 3 assessment videos. For each video, 3 edits will be made to the transcript and documented with their corresponding chunks for later reference.

Output: AFor each supported file format, a file is exported that contains every approved annotation for that assessment, with the same content as the edits made in the annotation interface.

Test Case Derivation: FR3 requires that clinicians can export annotation data from an assessment in a supported file format. Additionally, the file must contain all approved annotations for that assessment. This lets clinicians record, review, and share results without relying on other software.

How the test will be performed:

1. Make and approve the listed edits to the transcript in the annotation interface.
2. Export the annotations in one supported file format.
3. Open the exported file and check that every edit appears in it with the correct content.
4. Repeat steps 2 - 3 for each remaining supported file format.

3. FR-ST-DCS3: **Return Annotated Results to the Patient Record**

Purpose: Verifies that when a clinician finishes annotating a transcript, the system shows the number of verified and unverified segments, saves the annotated transcript to the patient's assessment record, and shows it with the assessment results.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the patient's assessment video is open in the annotation interface with its auto-generated transcript.

Input: 2 assessment videos where assessment 1 has all segments verified and assessment 2 has only 10 segments verified and the

remaining segments are left unverified.

Output: Before confirming, the system shows how many segments are verified and how many are not. After the clinician confirms, the system returns to the assessment's results and correctly shows the annotated transcript with the results and notes. For assessment 1, the transcript is marked as verified. For assessment 2, the transcript is saved and the unverified segments are indicated.

Test Case Derivation: UC10 requires that a finished annotation is saved to the patient's assessment record and shown with the assessment, and that unverified segments are indicated when annotation is completed without verifying everything. This lets clinicians keep track of which transcripts they have reviewed.

How the test will be performed:

1. For each video, verify the listed segments in the annotation interface and indicate that the annotation is finished.
2. Check that the counts of verified and unverified segments shown match what was verified, then confirm that the annotation is complete.
3. Check that the system returns to the assessment's results and shows the annotated transcript with the results and notes.
4. Check that assessment 1's transcript is marked verified, and that assessment 2's unverified segments are indicated.

4. FR-ST-DCS4: **Mark Patient Recording as Reviewed**

Purpose: Verifies that a clinician can mark a recording as reviewed, the review status is updated and shown, and that it persists when the recording is reopened.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, the patient's video assessment is open, and its recording has not been marked as reviewed.

Input: A patient assessment recording that has not been marked

as reviewed, including 2 notes made on separate questions while reviewing.

Output: Once the recording is marked as reviewed, the system shows the updated review status. When the assessment is reopened later, it is still shown as reviewed and the notes are unchanged.

Test Case Derivation: UC14 requires that the system updates and displays a recording's review status once the clinician marks it as reviewed. This helps clinicians tell which recordings they have finished reviewing and which are ready to be processed further.

How the test will be performed:

1. Open the recording in the Individual Assessment View, watch it, and make 2 notes.
2. Select the option to mark the recording as reviewed.
3. Check that the recording is now shown as reviewed.
4. Leave the Individual Assessment View, then reopen the same recording and check that it is still shown as reviewed and the notes have not changed.

4.1.3 Video and Audio Analysis

This section verifies that assessment videos are transcribed, speakers can be told apart, and interference and language errors are flagged.

1. FR-ST-VAA1: **Auto-Generate Transcript for Video Assessments**

Purpose: Verifies that the system automatically generates a transcript for assessment videos without clinician intervention.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and all assessment videos are in the system.

Input: 10 assessment videos.

Output: For each video, a non-empty transcript is generated and available for the clinician to review. At least

GENERATE_SUCCESS% of the videos must produce a transcript (NFR17).

Test Case Derivation: NFR1 requires that a transcript is generated for every supported video without any action from the clinician, and (NFR17) sets the required success rate. Additionally, a working auto-generation transcript tool is critical to streamlining the SLP workflow. **How the test will be performed:**

1. For each assessment video, open it in the annotation tool, and generate a transcription
2. Record whether each video's transcript was generated and appears in the annotation interface
3. Compute the success rate and compare it to GENERATE_SUCCESS%.

2. FR-ST-VAA2: **Distinguish Speakers in Transcripts**

Purpose: Verifies that the system visually distinguishes between speakers in a transcript, so the clinician can tell which speaker said each utterance.

Type: Manual, Dynamic

Initial State: A clinician is logged in, a patient record exists, and a transcript has already been generated for each assessment video.

Input: 10 assessment videos each containing at least two speakers. For each video, an answer key will be created by the team prior to testing by watching the video assessment. The answer key will contain the true speakers at 5 different timestamps.

Output: In every auto-generated transcript, each identified speaker is shown with a distinct label, tag, or highlight. For each of the 5 timestamps in the answer key, the label displayed by the system on that utterance must correspond to that on the answer key.

Test Case Derivation: FR4 requires that each speaker be distinguishable and that the clinician can tell which speaker corresponds to each utterance. Clinicians use this to annotate the right

individual and account for outside interferences such as a parent prompting the child. **How the test will be performed:**

1. For each assessment video, open it in the annotation tool, and generate a transcription
2. Record whether each video's transcript was generated and appears in the annotation interface
3. Compute the success rate and compare it to GENERATE_SUCCESS%.

3. FR-ST-VAA3: **Flag Potential Interference in Assessments**

Purpose: Verifies that the system flags questions where potential interference may have occurred and marks them visibly in the Individual Assessment View.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and the assessment video is already in the system.

Input: A video assessment containing 3 moments of interference (e.g., a guardian giving a hint during a question). A list of the question numbers where interference occurs is created by watching the video before testing.

Output: In the Individual Assessment View, every question on the list is flagged by the system.

Test Case Derivation: FR13 requires that each question with potential interference is flagged. Interference can make results invalid, so clinicians need to see which questions are affected.

How the test will be performed:

1. Open the Individual Assessment View for the assessment.
2. For each question number on the list, check that the question is flagged by the system.
3. If all listed questions are flagged, the test passes.

4. FR-ST-VAA4: **Show Grammar and Phonology Errors as Sug-**

gestions

Purpose: Verifies that the system identifies potential grammatical and phonological errors in a transcript and presents them to the clinician as unverified suggestions.

Type: Manual, Dynamic

Initial State: A clinician account is logged in, a patient record exists, and a transcript has been generated for each assessment video.

Input: 5 assessment videos in English and/or Mandarin, and each containing known grammatical and phonological errors. For each video, an answer key lists the timestamp and type (grammar or phonology) of each known error, created by the team before testing by watching the video.

Output: Each known error in the answer key is shown to the clinician as a suggestion linked to its transcript segment and marked as unverified. An error appears in the assessment report only after the clinician accepts it, and not if it is rejected or left unreviewed.

Test Case Derivation: FR26 requires that detected errors are linked to their transcript segment and shown as unverified until the clinician accepts or rejects them. Ambiguous speech can be clinically meaningful, so the system may only suggest errors and the clinician must make the final decision.

How the test will be performed:

1. Open each video's transcript in the annotation interface.
2. For each error in the answer key, check that the system shows it as a suggestion on the matching transcript segment, marked as unverified.
3. Accept some suggestions and reject others, then generate the assessment report and check that only the accepted errors appear in it.

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Area of Testing1

Title for Test

1. test-id1

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

Output:

How test will be performed:

4.2.2 Accuracy

This section verifies that the system is accurate enough for clinicians to rely on when reviewing assessments.

1. NFR-ST-AC1: **Flag Accuracy**

Purpose: Verifies that the system's interference flags are accurate enough for clinicians to rely on.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, a patient record exists, and the assessment videos are already in the system.

Input: 3 assessment videos, 2 of which contain interference (e.g., a guardian giving a hint during a question) and the remaining contain none. For each video, an answer key lists the question number and timestamp of each interference, created by the team before testing by watching the assessment.

Output: In the Individual Assessment View, every question with detected interference is visibly marked. Additionally, the system must:

- Flag at least FLAG_DETECTION_RATE% of the moments the answer key identifies as an interference
- Have no more than FLAG_FALSE_ALARM_RATE% of its flags as false alarms
- Place each flag within FLAG_TIME_TOLERANCE seconds of when the interference occurs in the video.

How the test will be performed:

1. For each video, open the Individual Assessment View.
2. Compare the flagged questions against the answer key of true interference moments. Count a moment as detected if its

question is flagged and the flag is within FLAG_TIME_TOLERANCE seconds of the timestamp listed in the answer key, and as missed otherwise. Any flag that is not listed in the answer key counts as a false alarm.

3. Compute the detection rate and false alarm rate over all videos and compare them to FLAG_DETECTION_RATE% and FLAG_FALSE_ALARM_RATE%.

4.2.3 Reliability

This section verifies that the system reliably generates transcripts for assessment videos and keeps clinicians' saved edits and annotations across sessions. This allows clinicians to depend on the system and ensure that their work is not lost.

1. NFR-ST-RE1: **Reliable Transcript Generation**

Purpose: Verifies that the system generates transcripts for assessment videos without the clinician having to restart the process manually.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, a patient record exists, and no transcript has been generated for the existing assessment videos.

Input: 20 valid assessment videos.

Output: For at least GENERATE_SUCCESS% of the 20 videos, a transcript is generated as specified in FR1 without the clinician having to restart the process manually NFR17.

How the test will be performed:

1. For each video assessment, open it up in the annotation interface and auto-generate a transcription.
2. When processing finishes, record whether a transcript was generated.
3. Repeat steps 1 - 2 for the remaining videos.

4. Compute the success rate (videos with a transcript / 20) and compare it to GENERATE_SUCCESS%.

2. NFR-ST-RE2: **Preserve Saved Edits and Annotations Across Sessions**

Purpose: Verifies that saved transcription edits and annotations remain available and unchanged after each clinician session.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, a patient record exists, and the assessment video is open in the annotation interface with its auto-generated transcript.

Input: 3 assessment videos where for each, 5 transcription edits and 5 annotations are made and saved (30 items in total). Each item is documented with the transcript chunk it was made on and its exact content for reference.

Output: For at least SAVE_SUCCESS% of the 30 saved items, the item is still shown and unchanged after the clinician logs out, logs back in, and reopens the assessment NFR18.

How the test will be performed:

1. For each video, make the listed edits and annotations and save them.
2. Log out of the clinician account, then log back in.
3. Reopen the assessment and check each listed item then record whether it is still shown and remains unchanged.
4. Compute the success rate (items shown unchanged / 30) and compare it to SAVE_SUCCESS%.

4.2.4 Recording Quality

The following test case verifies that all recording quality for speech video assessments are met

1. NFR-ST-P1: **Only Accept Videos Meeting Minimum Quality**

Purpose: Verifies that the system does not allow an assessment to proceed when the video resolution or audio level is below MIN_VIDEO_RESOLUTION or MIN_AUDIO_LEVEL, respectively.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in a patient record exists.

Input: 3 attempts to start an assessment, each with the camera and microphone set up as follows:

- Attempt 1: video resolution below MIN_VIDEO_RESOLUTION, audio at or above MIN_AUDIO_LEVEL.
- Attempt 2: audio level below MIN_AUDIO_LEVEL, video resolution at or above MIN_VIDEO_RESOLUTION.
- Attempt 3: video resolution exactly MIN_VIDEO_RESOLUTION and audio level exactly MIN_AUDIO_LEVEL.

Output: In Attempts 1 and 2 the assessment cannot proceed, and the system shows the reason (resolution too low for attempt 1, audio level too low for attempt 2). In Attempt 3 the assessment proceeds NFR10.

How the test will be performed:

1. Set the camera and microphone as described for Attempt 1, try to start the assessment, and record whether it proceeds and the reason shown if it does not.
2. Repeat step 1 for Attempts 2 and 3.
3. Check that Attempts 1 and 2 are blocked with the correct reason, and that Attempt 3 proceeds.

4.2.5 Maintainability

This section verifies that patient information and assessment recordings are backed up regularly and can be restored from a backup, so that clinicians' work is not lost.

1. NFR-ST-M1: **Regular Backups**

Purpose: Verifies that the system backs up patient information and assessment recordings, including recent changes, within one BACKUP_INTERVAL.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, a patient record exists, and the system's backups are running.

Input: One mock video assessment conducted on the system, with one edit made to one of its annotations and saved.

Output: After one BACKUP_INTERVAL, a backup for the mock assessment recording exists, including the edited annotation for the assessment transcript.

How the test will be performed:

1. Conduct the mock video assessment on the system.
2. Edit one annotation, save it, and note the time.
3. Wait one BACKUP_INTERVAL.
4. Open the backup storage and check that a backup exists and contains the recording and the edited annotation.

2. NFR-ST-M2: **Restore from Backup**

Purpose: Verifies that patient information and assessment recordings can be restored from a backup and match the original data.

Type: Nonfunctional, Dynamic, Manual

Initial State: A clinician account is logged in, and a backup exists that contains the mock assessment from NFR-ST-M1.

Input: The backup from NFR-ST-M1, and the mock assessment deleted from the system before restoring.

Output: After restoring from the backup, the mock assessment's recording and edited annotation are the same as before the deletion

NFR9.

How the test will be performed:

1. Open the mock assessment and document the recording length and the exact text of the edited annotation.
2. Delete the mock assessment from the system.
3. Restore the system from the backup.
4. Open the restored assessment and check that the recording plays and has the same length, and that the annotation text is the same as what was documented in step 1.

4.2.6 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software,

but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input/Condition:

   Output/Result:

   How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Javayria Mudassar, Catherine Yang, Yilin Jiang, and Nour Fawaz. System requirements specification for telac. https://github.com/Nour-Fawaz/MacaBoca, 2026.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

Variable Name	Value
MIN_VIDEO_RESOLUTION	640 x 480 pixels
MIN_AUDIO_LEVEL	-30 dBFS average
GENERATE_SUCCESS	95
SAVE_SUCCESS	100
FLAG_DETECTION_RATE	80%
FLAG_FALSE_ALARM_RATE	20%
FLAG_TIME_TOLERANCE	2 seconds
BACKUP_INTERVAL	24 hours

Table 1: Variable Names and Values

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

This section documents all the uses of generative artificial intelligence used throughout the document.

7.1 AI Tools Used

- Claude Sonnet 5 (Medium)
- ChatGPT-5.6 Luna

7.2 How and Where Used

The following lists the uses of AI applied throughout the entire document:

- Fixing grammar and spelling
- Debugging possible LaTeX errors
- Performing a final check over the document at the end, comparing each section against the original document comments and the rubric to ensure nothing was missed
- Creating references for the bibliography
- Removing template comments

Section 2:

- To create commands to link the requirements to the SRS and Hazard Analysis document

Section 4:

- Brainstorming test cases
- Splitting the test cases among the group members
- Creating the traceability matrix

7.3 Verification of AI-Generated Content

To verify the LLM's output, each team member went over their own section and presented it to the group, explaining what they had completed since the previous meeting. Every member was able to explain their section successfully. Before any pull request was approved, a team member who had not written the section also reviewed it to ensure it was cohesive, accurate, and consistent with the rest of the document. Since the document contained no factual content, no additional fact-checking of the LLM's output was needed.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.
4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Nour

Question 1

I wrote the Summary section of this deliverable. At this point, it's very clear what the objectives and core components of the project are. Hence, breaking

it down and determining how it maps back to our SRS was not difficult and was rather automatic. Additionally, considering our project builds on a previous year's capstone, we are using several of the same tools that they used. Hence, it was straightforward as to what frameworks will be needed for automated testing and verification. Moreover, when creating the test cases for the nonfunctional requirements, it was simple to divide them into different "Areas of Testing". This is thanks to the template that we used for the SRS document. It broke down the nonfunctional requirements into subsections making it clear which aspects regarding nonfunctional requirements needed testing.

Question 2

A pain point that the team experienced was splitting the functional requirements into different sections. As mentioned previously, the SRS template encouraged us to organize the nonfunctional requirements into subsections, however, it did not for the functional requirements. At the time this didn't seem like much of a problem, but it did make splitting the FRs into disjoint sections a bit difficult. A lot of the functional requirements map back to one another, so ensuring that the sections did not overlap was difficult. However, given that we are now more familiar with the system, especially after writing the SRS document and creating different components for risk mitigation in the Hazard Analysis document, it wasn't too difficult, mainly just time consuming. They were organized during a team meeting to ensure we all had a similar understanding of the system components and that we were on the same page about the main functions of the system.

Question 3

There are several different aspects that each team member has to be knowledgeable about to ensure we are able to successfully complete the verification and validation aspect of the project. I will personally tackle usability testing. This is because I have prior experience in this domain of testing. First, I had a couple of friends conduct usability tests on a game I created to detect any bugs and to determine what type of actions a user might take that may not come to mind when I am testing the game myself. This actually helped me catch a bug in the game, which I was later able to fix and update in the latest release. Second, I previously took the course SFWRENG 4HC3 which

is the Human Computer Interfaces course at McMaster. In that course, I ran card sorts, online usability interviews, and Figma prototype interviews where I observed users and asked several appropriate questions that avoided any biased wording. However, what I still need to build is applying these methods to a clinical setting, writing realistic SLP tasks, and using a standard questionnaire such as the System Usability Scale to produce scores we can compare against our requirements.

Question 4

The first approach I will use is reviewing and adapting my previous course material for this new application. Although I am familiar with usability testing, it has been a while, and the domains in which I applied the skills are vastly different from this one. Hence, I will review my 4HC3 notes and adapt the interview and observation methods I already used to this project.

The second approach is running mock interview sessions with friends and peers to ensure my wording and intent are clear. I will do this before involving any SLPs because, first there will not be many opportunities to ask the SLPs their opinions about certain features of TeLaC. Second, I do not want to waste their time by asking them questions that are hard to understand and are unclear. Ensuring that anyone can understand the point I am getting across will allow the clinician to quickly comprehend the question and prepare a well-thought-out answer.

I will pursue both approaches. This is because the first approach builds on methods I've already used, which is the fastest way to be effective. As for the second approach, it allows me to practice my interview skills at low risk and, because I have not conducted any usability tests with actual clinicians, it's a good way to develop my skills before being thrown into that setting.